docs_chain

No tracked document can fall out of sync — content-hashed, bidirectional, atomic.

Summary

Docs Chain is a universal, Go-implemented bidirectional document-and-database dependency-propagation engine. When any member of a registered chain changes — Markdown source, an HTML/PDF/DOCX export, or a SQLite database — it detects the change by content hash and propagates it through every connected member atomically.

Short description

A Go engine that keeps documents and databases in sync. Using Salsa-style content-hashed incremental recomputation over a DAG (Kahn topological ordering, early cutoff, bidirectional sync edges, atomic-rename + SQLite-transaction commits), it regenerates exports whenever any linked artifact changes.

Long description

Docs Chain is what you build once you have written the same fragile “regenerate the PDF when the Markdown changes” shell script one time too many. It replaces that entire genre of hand-rolled sync glue with a real engine. It models a project’s documents and databases as members of a chain and, when any member changes, propagates that change through every connected member in every declared direction — regenerating and re-exporting atomically so no tracked artifact can drift out of sync, ever. The design borrows its rigor straight from the world of incremental build systems rather than from scripting: change detection is by **content hash, not mtime**, so a `touch` triggers nothing and a one-byte edit triggers exactly the rebuilds it should — no false alarms, no missed changes. Stated formally in a single line, it is Salsa-style content-hashed incremental recomputation over a DAG, with Kahn topological ordering, early cutoff that

prunes unchanged subtrees, declared-authority bidirectional `sync` edges, and atomic-rename plus SQLite-transaction commits so a crash mid-propagation can never leave a half-written export behind. It ships as a `vasic-digital` submodule and is consumed as a core part of the HelixConstitution submodule, so any project that adopts the constitution gets Docs Chain out of the box and registers its own chains via per-context YAML. The implementation is honest about status (per constitution §11.4.6): Phases 1–4 (core DAG + hashing, node adapters/transforms, propagation orchestrator with atomicity, config-driven multi-context CLI with `sync`/`verify`/`doctor`/`graph`/`watch`) are implemented and tested; a Phase-4b adds generic bidirectional `md-to-sqlite`/`sqlite-to-md` builtins (pure-Go, row-level drift, byte-stable round-trip) and a `colorize-html` builtin; Phase 5 comprehensive real-binary e2e is implemented and GREEN. Phases 6–7 (constitution distribution, ATMOSphere wiring) remain PLANNED and operator-gated. Herald is the first real downstream consumer, syncing a 66-document multi-format corpus that verifies clean.

Why we built it

Documentation, exports, and databases drift apart the moment they're maintained by hand or by fragile scripts. Docs Chain makes synchronization mechanical, content-hash-accurate, and atomic, so a change anywhere in a chain correctly and safely updates everything downstream (and upstream).

Why it's a game-changer

It takes the hard-won correctness guarantees that compiler and build-system authors take for granted — content-hashed dependency graphs, minimal recomputation, atomic commits — and points them at documentation and databases, a domain that has historically limped along on cron jobs and good intentions. True bidirectional sync means the relationship between a source and its export is enforced in both directions, so "the docs are out of date" and "the export doesn't match the source" cease to be recurring bugs and become states the engine will not allow to exist.

What's innovative

- Content-hash (not mtime) incremental recompute over a DAG with early cutoff.
- Bidirectional, declared-authority sync edges (docs ↔ exports ↔ SQLite).
- Atomic-rename + SQLite-transaction commit for crash-safe propagation.

- Pure-Go `md-to-sqlite`/`sqlite-to-md` round-trip with row-level drift detection.

Challenges & solutions

- **Spurious rebuilds:** solved by content-hash detection instead of timestamps.
- **Partial/corrupt updates:** solved with atomic-rename and SQLite transactions.
- **Correct multi-member ordering:** solved with Kahn topological ordering + early cutoff.
- **Honest capability reporting:** solved by marking each phase IMPLEMENTED vs PLANNED per §11.4.6.

Tech stack (why + how)

- **Go** — entire engine (`internal/hash`, `graph`, `adapter`, `orchestrator`, `config`, `state`, `runner`, `cmd/docs_chain`).
- **DAG + Kahn topological sort** — dependency ordering with early cutoff.
- **SQLite (pure-Go modernc)** — DB members and transactional commits.
- **fsnotify** — `watch` daemon for live propagation.
- **YAML config** — per-context chain registration.
- **exec: transforms** — pluggable Markdown→HTML/PDF/DOCX generation.

> Roadmap honesty: Phases 6–7 (constitution distribution, ATMOSphere wiring) are PLANNED / operator-gated — not shipped.